Apple Green — Integração WhatsApp

Oct 21, 2026 · @Joao Gondim

Visão geral

O Apple Green já tem pronta toda a lógica de negócio do WhatsApp; falta só o canal. O integrador conecta o número do WhatsApp Business ao Apple Green por **quatro rotas HTTP** e cuida do que é próprio do WhatsApp: receber, transcrever áudio, enviar e manter os templates aprovados na Meta.

O que a integração entrega ao usuário final:

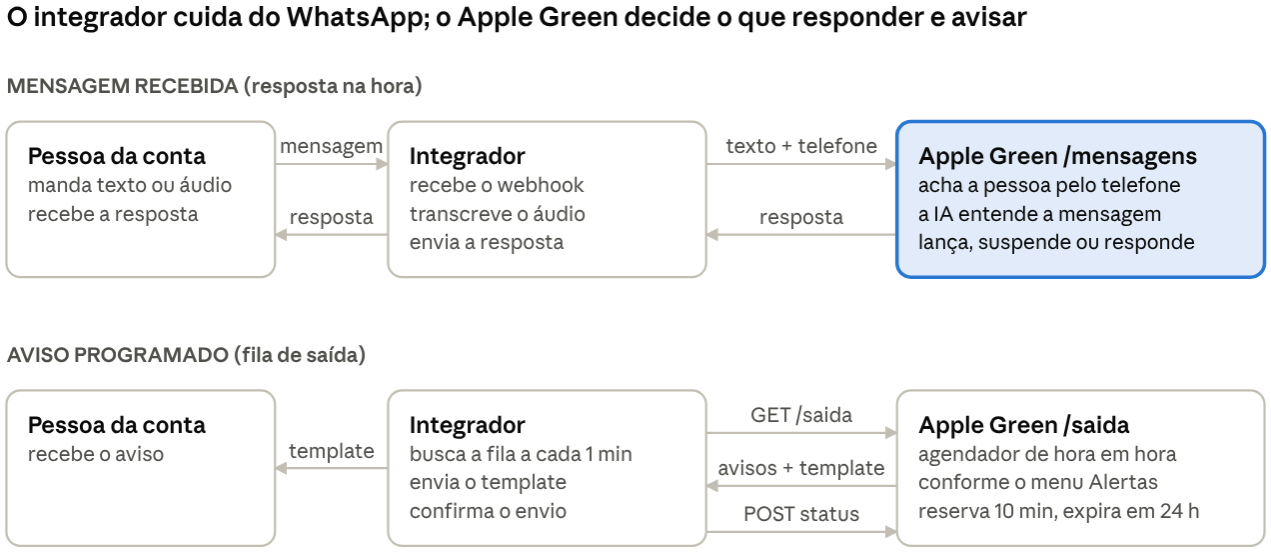
- **Lançar gastos por texto ou áudio**, sem informar categoria ou item ("almoço 45", "uber 23 ontem", "tv 1.200 em 10x"). O Apple Green classifica, lança e responde com o saldo do item; o que não entende fica em *Em suspenso* no app.
- **Perguntar sobre o orçamento**: "quanto ainda posso gastar com restaurante?", "quanto gastei com roupas em setembro?".
- **Duas pessoas por conta**: cada uma escreve do próprio número e tudo cai no mesmo orçamento.
- **Receber alertas programados**: contas a vencer, fatura, meta perto do limite, quanto ainda dá para gastar no fim de semana. O usuário escolhe quais no menu **Alertas** do app.

Responsabilidade	Integrador	Apple Green
Número WhatsApp Business, webhook da Meta/BSP	✓	
Baixar e transcrever áudios	✓	
Enviar a resposta de cada mensagem recebida	✓	
Buscar a fila de avisos e enviá-los como template	✓	
Cadastrar e aprovar os templates na Meta	✓	textos e variáveis definidos aqui
Identificar a pessoa pelo telefone		✓
Entender a mensagem, classificar, lançar, responder		✓

Responsabilidade	Integrador	Apple Green
Decidir quais alertas gerar, quando e para quem		✓
Guardar histórico, custos de IA e status de envio		✓

Arquitetura e fluxos

São dois fluxos independentes. A mensagem do usuário é respondida na hora, na mesma chamada HTTP. Os avisos programados ficam numa fila que o integrador busca.



arquitetura · 2 fluxos, 4 rotas

No primeiro fluxo o integrador só repassa: quem decide o que lançar e o que responder é o Apple Green. No segundo, o Apple Green decide o que avisar e para quem, e o integrador entrega e confirma.

Autenticação, ambientes e variáveis

Toda chamada às rotas `/integracao/whatsapp/*` leva o cabeçalho `Authorization: Bearer <WHATSAPP_INTEGRACAO_TOKEN>`. É um segredo compartilhado servidor a servidor: nunca vai para o navegador nem para o app do usuário.

- Token ausente ou errado → `401`. Variável não definida no servidor → `503` (integração desligada).
- Gere um token longo e aleatório (ex.: `openssl rand -hex 32`) e troque-o se vazar; não há rotação automática.

- Todas as rotas aceitam e devolvem JSON (`Content-Type: application/json`).

Variáveis do servidor do Apple Green (arquivo `backend/.env`):

Variável	Obrigatória	Para que serve
`WHATSAPP_INTEGRACAO_TOKEN`	sim	Liga as rotas da integração e a fila de saída. Sem ela os avisos ficam só no histórico do app.
`ANTHROPIC_API_KEY`	recomendada	Interpretação das mensagens por IA. Sem ela o WhatsApp só lança o que o usuário já ensinou ao app e não responde perguntas.
`WHATSAPP_MODELO_IA`	não	Modelo da interpretação. Padrão `claude-opus-5-5`; `claude-haiku-5-5` custa bem menos (ver custos).

URL base: a do backend em produção (ainda a definir). Localmente: `http://localhost:3001`.

Contrato da API

São quatro rotas. Em todas, erro de validação volta como `400` com `{"erro": "mensagem"}`.

Rota	Quando chamar	Resposta
`POST /integracao/whatsapp/mensagens`	A cada mensagem recebida de um usuário	Texto a enviar de volta
`GET /integracao/whatsapp/saida`	Periodicamente (sugestão: a cada 1 min)	Avisos prontos para envio
`POST /integracao/whatsapp/saida/:id/status`	Depois de tentar enviar cada aviso	`{"ok": true}`
`GET /integracao/whatsapp/saude`	Monitoramento e teste do token	Situação do serviço

POST /integracao/whatsapp/mensagens

Recebe uma mensagem e devolve, na mesma resposta HTTP, o texto que o integrador deve enviar a quem escreveu. Resposta síncrona; com IA leva em geral de 2 a 6 segundos.

```
{
  "mensagem_id": "wamid.HBgMNTU2Mjk5MjI0MjMzMBUCABIYFjNFQjA...",
  "telefone": "5562992242330",
  "tipo": "texto",
  "texto": "almoço 45",
  "recebida_em": "2026-10-21T15:04:11Z"
}
```

Campo	Obrigatório	Regra
`mensagem_id`	recomendado	Id da mensagem no WhatsApp (`wamid`). Garante que a mesma mensagem não seja processada duas vezes.
`telefone`	sim	Número de quem enviou, como o WhatsApp entrega (com ou sem 55, com ou sem o 9).
`tipo`	não	`texto` (padrão) ou `audio`.
`texto`	sim	O texto da mensagem; em áudio, a transcrição. Vazio em áudio = "não consegui entender".
`recebida_em`	não	Informativo.

Resposta `200`:

```
{
  "id": 128,
  "mensagem_id": "wamid.HBgMNTU2Mjk5MjI0MjMzMBUCABIYFjNFQjA...",
  "acao": "lancado",
  "intencao": "lancamento",
  "resposta": "✅ Lancei R$ 45 em Almoço / Lanches / Jantar (hoje).\nAlmoço / Lanches / Jantar: ainda pode gastar R$ 140,80 (usou R$ 82,20 da meta de R$ 223 do mês).",
  "detalhes": [{ "resultado": "lancado", "item_id": 175, "valor": 45, "data": "2026-10-21", "parcelas": 1 }]
}
```

- **Envie `resposta` exatamente como veio**, como mensagem de sessão (está dentro da janela de 24h aberta pelo usuário). Pode ter quebras de linha e emojis.
- `acao` serve para métricas e logs: `lancado`, `parcial`, `suspenso`, `consulta`, `consulta_sem_alvo`, `ajuda`, `numero_desconhecido`, `numero_ambiguo`, `audio_sem_transcricao`, `erro`, `recusa`.
- Número não cadastrado também volta `200`, com uma `resposta` explicando como se cadastrar; o integrador só envia.

- Mesma `mensagem_id` de novo → devolve a resposta já dada, com `"repetida": true`, sem lançar outra vez. Se a `resposta` já foi entregue, não reenvie.

GET /integracao/whatsapp/saida

Entrega os avisos programados prontos para envio e os **reserva por 10 minutos**: uma segunda leitura não os devolve de novo. Sem confirmação em 10 minutos, voltam para a fila. Parâmetro opcional `?limite=50` (máximo 100).


```
[
  {
    "id": 312,
    "usuario_id": 2,
    "pessoa_id": 9,
    "tipo": "meta_atingida",
    "telefone": "62992242330",
    "titulo": "Água: 86% da meta do mês",
    "mensagem": "⚠️ Água já usou 86% da meta do mês (R$ 85,90 de R$ 100).
Ainda dá para gastar R$ 14,10.",
    "template": "ag_meta_atingida",
    "variaveis": { "nome": "Maria", "item": "Água", "percentual": "86",
"realizado": "R$ 85,90", "meta": "R$ 100", "disponivel": "R$ 14,10" },
    "data_ref": "2026-10-21",
    "criado_em": "2026-10-21 11:00:02"
  }
]
```


- O `telefone` vem só com dígitos, como foi cadastrado (normalmente sem o 55): acrescente o código do país antes de enviar.
- Fora da janela de 24h, envie como **template** (`template` + `variaveis`). Dentro da janela, pode enviar o texto pronto em `mensagem` .

POST /integracao/whatsapp/saida/:id/status


```
{ "status": "enviado", "mensagem_id_provedor": "wamid.HBgM..." }
```

```
{ "status": "falhou", "erro": "131026: Message undeliverable" }
```


- `status` : `enviado` ou `falhou` . Um aviso `falhou` não volta sozinho para a fila; reenviar fica a critério do integrador, com nova confirmação.
- Confirmação repetida de um aviso já `enviado` → `404` .

GET /integracao/whatsapp/saude

```
{ "ok": true, "ia": true, "modelo": "claude-opus-5-5", "hora": "2026-10-21T13:39:58.017Z" }
```

`ia: false` significa que o servidor está sem `ANTHROPIC_API_KEY`: as mensagens ainda são recebidas, mas só as já aprendidas são lançadas.

Regras de funcionamento

Identificação pelo telefone. A pessoa é encontrada pelo telefone cadastrado em *Configurações › Pessoas*. A comparação usa DDD + últimos 8 dígitos, então `5562992242330`, `556292242330` e `(62) 99224-2330` são o mesmo número. O app impede o mesmo telefone em duas contas.

Duas pessoas por conta. Cada conta tem até 2 pessoas, cada uma com seu telefone. As duas lançam no mesmo orçamento; cada lançamento guarda quem lançou. Itens de gastos pessoais são atribuídos de preferência à pessoa que escreveu. Cada alerta pode ir para as duas ou só para uma, conforme o usuário configurar.

Áudio. O Apple Green não recebe arquivos de áudio: o integrador baixa a mídia, transcreve e envia o texto com `"tipo": "audio"`. A transcrição deve ser em português do Brasil; erros pequenos são tolerados. Opções e preços na seção de custos.

Idempotência. Envie sempre o `mensagem_id` do WhatsApp. Se a Meta reentregar o webhook ou houver retentativa por timeout, a segunda chamada devolve a mesma resposta (`"repetida": true`) sem lançar de novo.

Tempo de resposta e retentativas. Use timeout de 30 s na chamada a `/mensagens`. Em erro de rede ou `5xx`, tente de novo com o mesmo `mensagem_id`. Nunca reenvie ao usuário uma resposta já entregue.

Fila de saída.

1. O agendador do Apple Green roda de hora em hora, a partir das 8h (Brasília), e grava os avisos do dia: um por pessoa que recebe.
2. O integrador busca a fila; cada aviso entregue fica reservado por 10 minutos.
3. O integrador envia e confirma `enviado` ou `falhou`.
4. Aviso sem confirmação volta à fila depois de 10 minutos.
5. Aviso com mais de 24 h sem envio vira `expirado` e não é mais entregue, para não chegar fora de hora (ex.: "vence hoje" no dia seguinte).

Cada aviso tem uma chave única (tipo + item + data + pessoa): o mesmo aviso nunca é gerado duas vezes.

Lançamentos com outra pessoa avisada. Se o usuário ligar essa opção, quando um lança pelo WhatsApp o outro recebe um aviso na fila (`tipo: saldo_pos_lancamento`).

Templates da Meta

Avisos chegam quase sempre fora da janela de 24h, então cada tipo precisa de um template aprovado na categoria **Utility**, idioma `pt_BR` . O nome do template vem em `template` e os valores em `variaveis` ; a ordem dos parâmetros abaixo é a que o integrador deve usar.

Template	Tipo de alerta	Parâmetros, em ordem	Texto sugerido
`ag_conta_vencendo`	`conta_vencendo`	nome, conta, vencimento, valor	Olá, {{1}}! A conta {{2}} vence em {{3}}. Valor previsto: {{4}}.
`ag_fatura_fecha`	`fatura_fecha`	nome, cartao, fechamento	Olá, {{1}}! A fatura do {{2}} fecha em {{3}}. Compras feitas depois disso entram na fatura seguinte.
`ag_melhor_dia_compra`	`melhor_dia_compra`	nome, cartao, pagamento	Olá, {{1}}! Hoje é o melhor dia para comprar no {{2}}: a compra só será paga em {{3}}.
`ag_fatura_vence`	`fatura_vence`	nome, cartao, vencimento	Olá, {{1}}! A fatura do {{2}} vence em {{3}}.

Template	Tipo de alerta	Parâmetros, em ordem	Texto sugerido
`ag_meta_atingida`	`meta_atingida`	nome, item, percentual, realizado, meta, disponivel	Olá, {{1}}! {{2}} já usou {{3}}% do planejado: {{4}} de {{5}}. Saldo: {{6}}.
`ag_limite_programado`	`limite_programado`	nome, total, por_dia, mes	Olá, {{1}}! Até o fim de {{4}} ainda dá para gastar {{2}} nos itens que você escolheu, cerca de {{3}} por dia.
`ag_lancamento_outra_pessoa`	`saldo_pos_lancamento`	nome, pessoa, valor, item, saldo	Olá, {{1}}! {{2}} lançou {{3}} em {{4}}. Saldo do item: {{5}}.

- `nome` é sempre o primeiro nome de quem recebe.
- Valores já vêm formatados (`R$ 1.200` , `21/out`); não reformate.
- A Meta recusa parâmetro vazio ou com quebra de linha. As variáveis nunca vêm vazias nem com quebra de linha; o texto completo, com várias linhas, fica em `mensagem` .
- Os textos sugeridos podem ser ajustados na aprovação, desde que mantenham os mesmos parâmetros. Se mudar nome ou ordem, avise para alinharmos o código.

Como o Apple Green interpreta as mensagens

A IA só entende a mensagem; os valores das respostas sempre saem do banco de dados, nunca da IA. O integrador não precisa implementar nada desta seção — ela explica o que esperar das respostas.

1. **Atalho sem IA.** Se a mensagem tem valor, não parece pergunta e o termo já foi ensinado pelo usuário (ao confirmar um gasto em suspenso), lança direto. Custo zero.

2. **IA.** Caso contrário, o modelo recebe a mensagem, a data de hoje, quem escreveu e a lista de itens do orçamento daquela conta. Devolve em JSON: a intenção (lançamento, consulta ou outro), os gastos com valor, data, parcelas e item, e o grau de certeza.
3. **Lançamento.** Só lança quando o item é claro e há valor. Parcelado vira uma parcela por mês. O resto vai para *Em suspenso*, com o item sugerido, para o usuário confirmar no app. Uma mensagem pode ter vários gastos ("uber 23 e café 8").
4. **Consulta.** O sistema calcula pelos mesmos critérios da tela Gestão de Pagamentos: itens mensais = meta do mês − gasto no mês; itens eventuais = limite anual − a janela de 12 meses mais cheia.
5. **Outro.** Saudações e dúvidas recebem uma resposta curta com exemplos de uso.

Exemplos de resposta (texto em `resposta`):

Mensagem	`acao`	Resposta
almoço 45	`lancado`	✅ Lancei R$ 45 em Almoço / Lanches / Jantar (hoje). Almoço / Lanches / Jantar: ainda pode gastar R$ 140,80 (usou R$ 82,20 da meta de R$ 223 do mês).
comprei celular 1200 em 10x	`lancado`	✅ Lancei R$ 1.200 em 10x em Troca de Celular / Itens Eletrônicos (hoje).
presente 80	`suspenso`	⏸️ "presente" (R$ 80) ficou em suspenso: não tive certeza do item. Confira em Gestão de Pagamentos › Em suspenso.
quanto ainda posso gastar com alimentação?	`consulta`	💰 Alimentação fora de casa: ainda pode gastar R$ 140,80 em outubro.
quanto gastei com uber desde setembro?	`consulta`	📊 Gastos em setembro/2026 a outubro/2026: Uber / Taxi / 99Pop: R$ 0
oi	`ajuda`	Resposta curta explicando como lançar e consultar.

Tudo fica registrado no Apple Green: mensagem, ação, resposta, modelo, tokens e custo estimado de IA por mensagem.

Testes

Simulador no app. O menu *Alertas* tem a área "Testar conversa": escreve-se como no WhatsApp, escolhendo qual pessoa fala, e vê-se a resposta exata que a rota `/mensagens` devolveria. Os gastos lançados ali entram de verdade no orçamento.

Ambiente local.

```
cd backend && WHATSAPP_INTEGRACAO_TOKEN=dev-wpp-local DEV_SEM_LOGIN=1 node server.js
```

```
curl -s -X POST http://localhost:3001/integracao/whatsapp/mensagens -H "Authorization: Bearer dev-wpp-local" -H "Content-Type: application/json" -d '{"mensagem_id":"teste-1","telefone":"5562999999999","texto":"oi"}'
```

Para ver avisos na fila sem esperar o agendador (só local, com `DEV_SEM_LOGIN=1`), use `POST /avisos/simular?hoje=2026-10-23&hora=18&gravar=1` com o token de login do app.

Cenários de aceite.

- [ ] Texto com item claro ("almoço 45") → `lancado` e resposta com o saldo do item.
- [ ] Áudio transcrito → mesmo resultado do texto equivalente, com `tipo: audio`.
- [ ] Item incerto ("presente 80") → `suspenso`; aparece em *Em suspenso* no app.
- [ ] Parcelado ("tv 1.200 em 10x") → 10 parcelas de R$ 120 nos meses seguintes.
- [ ] Pergunta de saldo e de total gasto → `consulta` com os valores da tela Gestão de Pagamentos.
- [ ] As duas pessoas da conta lançam e os gastos caem no mesmo orçamento.
- [ ] Número não cadastrado → `numero_desconhecido` e resposta de orientação.
- [ ] Mesma `mensagem_id` duas vezes → a segunda volta `repetida: true` e não lança de novo.
- [ ] Fila: avisos entregues, reservados por 10 min, confirmados como `enviado`; `falhou` registra o erro.
- [ ] Cada template é enviado com os parâmetros na ordem da tabela e aparece certo no celular.

Estimativa de custos

Uma família ativa (2 pessoas) custa cerca de **US$ 2,50 a 3,30 por mês** com o modelo padrão (Claude Opus 5.5), ou **US$ 0,55 a 1,40** com o Claude Haiku 5.5. Cerca de 60% a 80% do custo com Opus é a IA; com Haiku, o maior custo passa a ser o WhatsApp.

Premissas por família, por mês: 120 mensagens recebidas (4 por dia), 30% delas em áudio de 10 s, todas interpretadas pela IA (sem o atalho de termos aprendidos, que não custa nada) e cerca de 70 avisos por template.

Item (US$ por família por mês)	Com Opus 5.5	Com Haiku 5.5	Base do cálculo
IA: interpretar 120 mensagens	1,97	0,05	~2.600 tokens de entrada e ~300 de saída por mensagem
Transcrição: 36 áudios, 6 min	0,02	0,02	~US$ 0,003/min (OpenAI gpt-4o-mini-transcribe)
WhatsApp: ~70 avisos por template	0,48	0,48	~US$ 0,0068 por template Utility no Brasil
WhatsApp: 120 respostas às mensagens	0 a 0,82	0 a 0,82	grátis até 1.000/mês por número; acima, tarifa a confirmar
Total aproximado	**2,47 a 3,29**	**0,55 a 1,37**	sem a taxa do provedor (BSP)

IA. Preços por milhão de tokens (entrada / saída): Opus 5.5 US$ 4 / 20, Sonnet 5.5 US$ 2 / 10 (cerca de US$ 1,00 por família por mês), Haiku 5.5 US$ 0,10 / 0,50. Mensagens seguidas em até 5 minutos aproveitam o cache da lista de itens e saem perto da metade do preço com Opus. A troca de modelo é só a variável `WHATSAPP_MODELO_IA`; recomendo testar o Haiku com mensagens reais antes de mudar, porque a qualidade da classificação não foi medida.

WhatsApp. Templates Utility no Brasil: cerca de US$ 0,0068 por mensagem; uma fonte indica US$ 0,0078 a partir de 1º de outubro de 2026. Desde essa data, a Meta passou a cobrar respostas de atendimento acima de 1.000 por mês por número, além de templates enviados dentro da janela de 24h. O aviso "a outra pessoa lançou" (opcional) soma cerca de US$ 0,40 por mês se a família lançar 60 gastos.

Transcrição. Alternativas: Whisper (`whisper-1`) ~US$ 0,006/min; Google Speech-to-Text V2 ~US$ 0,016/min, com 60 min grátis por mês. Em qualquer uma o custo é irrelevante perto do resto.

Fora da conta: a taxa do provedor do WhatsApp (BSP), que pode ser por mensagem ou mensalidade fixa, e a hospedagem.

Fontes: preços da IA da tabela oficial da Anthropic; WhatsApp e transcrição de levantamentos de terceiros, não das páginas oficiais — confirme na tabela de preços da Meta antes de fechar o orçamento. Levantamentos usados: EngageLab, Message Central, DIY AI — OpenAI, DIY AI — Google.

Como subir o sistema

O Apple Green é um frontend Angular estático mais um backend Node.js com banco SQLite em arquivo. Sobe em uma única máquina com disco persistente; não há banco externo, fila ou cache para provisionar.

Parte	Tecnologia	Como gerar e rodar	Onde hospedar
Frontend	Angular 22, TypeScript 6	`npm ci && npm run build` → arquivos em `dist/apple-green/browser`	Qualquer hospedagem estática com HTTPS e retorno de todas as rotas para `index.html` (Vercel, Netlify, Cloudflare Pages, S3 + CloudFront)
Backend	Node.js 24, Express 5, better-sqlite3	`cd backend && npm ci && npm start`	Servidor com disco persistente: VPS, Render com disco, Railway ou Fly.io com volume

O que o dono do produto entrega ao DEV

- [ ] Domínio (ex.: `app.` para o frontend e `api.` para o backend) e acesso ao DNS
- [ ] Chave da API da Anthropic (`ANTHROPIC_API_KEY`), em conta com cobrança ativa
- [ ] Credenciais OAuth do Google (`google_credentials.json`) da conta Gmail que envia os e-mails de confirmação de cadastro
- [ ] E-mail(s) que terão acesso master
- [ ] Acesso ao número do WhatsApp Business e ao provedor escolhido (BSP)
- [ ] Conta do gateway de pagamento, quando for escolhido

Frontend: única configuração

Em `src/environments/environment.prod.ts` , troque `apiUrl` pela URL pública do backend (hoje está o exemplo `https://SEU-BACKEND.onrender.com`). Mantenha `semLogin: false` .

Backend: variáveis de ambiente

Variável	Obrigatória	Valor em produção
`NODE_ENV`	sim	`production`

Variável	Obrigatória	Valor em produção
`PORT`	não	Porta do processo (padrão `3001`)
`FRONTEND_URL`	sim	URL do frontend, sem barra no fim. Libera o CORS e monta os links dos e-mails.
`JWT_SECRET`	sim	Segredo aleatório com 32 caracteres ou mais (ex.: openssl rand -hex 32). Com NODE_ENV=production, o servidor não sobe sem ele. Trocar derruba todas as sessões.
`DATA_DIR`	sim	Pasta no disco persistente onde ficam banco, imagens enviadas e credenciais do Google
`ANTHROPIC_API_KEY`	recomendada	Leitura de faturas em PDF e interpretação das mensagens do WhatsApp
`WHATSAPP_INTEGRACAO_TOKEN`	para o WhatsApp	Token da integração (ver Autenticação)
`WHATSAPP_MODELO_IA`	não	`claude-opus-5-5` (padrão) ou `claude-haiku-5-5`
`MASTER_EMAILS`	sim	E-mails com acesso à área master, separados por vírgula
`PAGAMENTOS_PROVEDOR`	não	`manual` até o gateway ser integrado

Nunca defina `DEV_SEM_LOGIN` em produção: ela permite entrar sem senha. Com NODE_ENV=production, o servidor se recusa a subir se ela estiver ligada.

Dados que precisam persistir (`DATA_DIR`)

- `apple-green.db` (com `-wal` e `-shm`): o banco inteiro. Tabelas e colunas são criadas e atualizadas sozinhas quando o servidor sobe; não há migração manual.
- `uploads/`: foto do anúncio do planejador e imagens dos avisos.
- `google_credentials.json`: colocado pelo DEV. `google_token.json` é gerado ao autorizar o Gmail.

Faça backup diário dessa pasta (ex.: `sqlite3 apple-green.db ".backup backup.db"` ou snapshot do volume) e guarde fora do servidor.

Regras de execução

- **Uma única instância do backend.** Os agendadores (avisos de hora em hora e assinaturas) rodam dentro do processo, e o SQLite não aceita vários servidores escrevendo no mesmo arquivo. Nada de escalar horizontalmente.
- Processo sempre no ar, com reinício automático em falha (pm2, systemd ou o gerenciador do provedor).
- HTTPS no frontend e no backend. O backend precisa ser público para o app, para o integrador do WhatsApp (`/integracao/whatsapp/*`) e para o gateway de pagamento (`/pagamentos/webhook/:provedor`).
- O fuso do servidor não importa: as regras usam o horário de Brasília internamente.
- Logs do processo: requisições lentas e falhas dos agendadores saem no console.

Primeira subida, passo a passo

1. Criar o servidor e o volume persistente; definir as variáveis de ambiente.
2. Subir o backend e conferir no log a linha "API rodando na porta".
3. Ajustar `environment.prod.ts`, gerar o build e publicar o frontend.
4. Colocar `google_credentials.json` no `DATA_DIR`, entrar no app e abrir `/auth/google?token=<token da sessão>` no backend para autorizar o envio de e-mails.
5. Criar a conta do dono pelo cadastro do app, com um e-mail de `MASTER_EMAILS`, e confirmar que o menu Master aparece.
6. No menu Financeiro, criar o plano de assinatura; no menu Anúncios e avisos, revisar o anúncio do planejador e enviar a foto.
7. Passar ao integrador do WhatsApp a URL do backend e o `WHATSAPP_INTEGRACAO_TOKEN`.

Checklist de go-live

- [ ] Endereço do backend de produção definido e com HTTPS
- [ ] `WHATSAPP_INTEGRACAO_TOKEN` gerado e guardado no servidor e no integrador
- [ ] `ANTHROPIC_API_KEY` no servidor e modelo escolhido (`WHATSAPP_MODELO_IA`)
- [ ] Número do WhatsApp Business verificado e webhook ligado ao serviço do integrador
- [ ] Transcrição de áudio em português funcionando
- [ ] Os 7 templates aprovados na Meta com os parâmetros na ordem da tabela
- [ ] Busca da fila a cada 1 minuto e confirmação de cada envio
- [ ] Telefones das pessoas cadastrados em *Configurações* do app

- [ ] Cenários de aceite da seção Testes aprovados
- [ ] Monitoramento de `/saude` e de taxa de erros de envio
- [ ] Termo de uso e consentimento (opt-in) para receber mensagens pelo WhatsApp